

Lycée Technique IBN SINA — Kénitra
Module M10 · Algorithmique et Programmation

Mini-Projet en Langage C

Gestion d'un Stock de Produits
avec Listes Chaînées

Rapport Technique
Année scolaire 2025/2026

DAI Première Année

1. Objectif du Projet

Ce mini-projet consiste à développer une application complète en langage C permettant de gérer un stock de produits à l'aide de **listes chaînées dynamiques**. L'objectif pédagogique est de maîtriser les structures de données dynamiques, la gestion de la mémoire avec malloc et free, ainsi que la modularité du code via des fonctions bien définies.

Le programme final couvre la saisie, l'affichage, la recherche, l'insertion, la suppression, la modification des produits, ainsi que la persistance des données via des fichiers binaires.

2. Définition des Structures (Partie 1)

L'architecture repose sur trois structures *typedéfinies*, déclarées en amont pour résoudre les problèmes d'auto-référencement :

2.1 Forward Declarations

```
typedef struct produit produit;  
typedef struct maillon maillon;  
typedef struct stock stock;
```

2.2 Structure produit

```
struct produit {  
    char code[10]; // Identifiant unique  
    char nom[50];  // Nom du produit  
    int qte;       // Quantité en stock  
    float prix;    // Prix unitaire  
};
```

2.3 Structure maillon

```
struct maillon {  
    produit p; // Données du produit  
    maillon* suivant; // Pointeur vers le nœud suivant  
};
```

2.4 Structure stock

```
struct stock {  
    maillon* tete; // Pointeur vers le premier maillon  
};
```

■ **Note:** Le choix des forward declarations permet d'utiliser maillon* à l'intérieur de struct maillon sans ambiguïté de compilation.

3. Gestion des Produits (Partie 2)

3.1 lireProduit()

Saisit les informations d'un produit depuis l'entrée standard. Utilise `fgets` pour les chaînes afin de supporter les espaces, et `getchar()` pour vider le buffer après `scanf`.

```
produit lireProduit() {
    produit p;
    printf("Code: "); scanf("%s", p.code); getchar();
    printf("Nom: "); fgets(p.nom, sizeof(p.nom), stdin);
    p.nom[strlen(p.nom)-1] = '\0';
    printf("Qte: "); scanf("%d", &p.qte); getchar();
    printf("Prix: "); scanf("%f", &p.prix); getchar();
    return p;
}
```

■ **Note:** `p.nom[strlen(p.nom)-1] = '\0'` remplace le saut de ligne laissé par `fgets` par un terminateur null.

3.2 afficheProduit()

```
void afficheProduit(produit p) {
    printf("|%s\t|%s\t|%d\t|%.2f\t|\n",
        p.code, p.nom, p.qte, p.prix);
}
```

3.3 valeurProduit()

```
float valeurProduit(produit p) {
    return p.prix * p.qte;
}
```

4. Gestion du Stock (Partie 3)

4.1 creerMaillon()

Alloue dynamiquement un nouveau nœud et l'initialise. Toujours initialiser suivant à NULL pour éviter les pointeurs pendants.

```
maillon* creerMaillon(produit p) {
    maillon* m = (maillon*)malloc(sizeof(maillon));
    m->p = p;
    m->suivant = NULL;
    return m;
}
```

4.2 initStock() et codeExiste()

```
void initStock(stock *s) {
    s->tete = NULL;
}

bool codeExiste(stock s, char code[]) {
    maillon* ptr = s.tete;
    while (ptr != NULL) {
        if (!strcmp(ptr->p.code, code)) return true;
        ptr = ptr->suivant;
    }
    return false;
}
```

4.3 rechercheProduit()

Retourne un pointeur direct vers le produit dans le *heap* — valide tant que le nœud existe.

```
produit* rechercheProduit(stock s, char code[]) {
    maillon* ptr = s.tete;
    while (ptr != NULL) {
        if (!strcmp(ptr->p.code, code)) return &(ptr->p);
        ptr = ptr->suivant;
    }
    return NULL;
}
```

4.4 Ajout en début et en fin

```
void ajouterProduitDebut(stock* s, produit p) {
    if (codeExiste(*s, p.code)) { printf("Erreur\n"); return; }
    maillon* m = creerMaillon(p);
    m->suivant = s->tete;
    s->tete = m;
}

void ajouterProduitFin(stock* s, produit p) {
    if (codeExiste(*s, p.code)) { printf("Erreur\n"); return; }
    if (s->tete == NULL) { s->tete = creerMaillon(p); return; }
    maillon* ptr = s->tete;
    while (ptr->suivant != NULL) ptr = ptr->suivant;
    ptr->suivant = creerMaillon(p);
}
```

4.5 Suppression

```
void supprimerDebut(stock* s) {
    if (!s->tete) return;
    maillon* tmp = s->tete->suivant;
    free(s->tete);
    s->tete = tmp;
}

void supprimerFin(stock* s) {
    if (!s->tete) return;
    if (!s->tete->suivant) { free(s->tete); s->tete = NULL; return; }
    maillon* ptr = s->tete;
    while (ptr->suivant->suivant != NULL) ptr = ptr->suivant;
    free(ptr->suivant);
    ptr->suivant = NULL;
}
```

4.6 Modification

```
void modifierProduit(stock* s, char code[]) {
    produit* p = rechercheProduit(*s, code);
    if (!p) return;
    /* ... Saisie des nouveaux champs ... */
    strcpy(p->nom, nouveau_nom); p->qte = n_qte; p->prix = n_prix;
}
```

5. Affichage et Calculs

5.1 afficherStock()

```
void afficherStock(stock s) {
    maillon* ptr = s.tete;
    while (ptr != NULL) { afficheProduit(ptr->p); ptr = ptr->suivant; }
}
```

5.2 afficherRuptureStock()

Filtre uniquement les produits dont la quantité est égale à 0.

```
void afficherRuptureStock(stock s) {
    maillon* ptr = s.tete;
    while (ptr != NULL) {
        if (ptr->p.qte == 0) afficheProduit(ptr->p);
        ptr = ptr->suivant;
    }
}
```

5.3 totalStock()

■ **Note:** Toujours initialiser `sum` à 0 — une variable locale non initialisée contient une valeur indéfinie (garbage).

```
float totalStock(stock s) {
    maillon* ptr = s.tete;
    float sum = 0;
    while (ptr != NULL) {
        sum += valeurProduit(ptr->p);
        ptr = ptr->suivant;
    }
    return sum;
}
```

5.4 produitLePlusCher()

Retourne un pointeur vers le produit le plus cher directement dans le heap. Le pattern *NULL-guard* sur la première itération évite toute comparaison invalide.

```
produit* produitLePlusCher(stock s) {
    maillon* ptr = s.tete;
    produit* prdMax = NULL;
    while (ptr != NULL) {
        if (prdMax == NULL || ptr->p.prix > prdMax->prix)
            prdMax = &ptr->p;
        ptr = ptr->suivant;
    }
    return prdMax;
}
```

6. Persistance des Données (Fichiers Binaires)

La persistance est assurée par des fichiers binaires. On ne sauvegarde pas la structure `stock`, mais les données produit nœud par nœud.

6.1 sauvegarderStock()

```
void sauvegarderStock(stock s, char nomFichier[]) {
    FILE* fp = fopen(nomFichier, "wb");
    maillon* ptr = s.tete;
    while (ptr != NULL) {
        fwrite(&ptr->p, sizeof(produit), 1, fp);
        ptr = ptr->suivant;
    }
    fclose(fp);
}
```

6.2 chargerStock()

La lecture exploite la valeur de retour de `fread` : elle retourne le nombre d'éléments lus, soit 1 tant qu'un produit complet est lu.

```
void chargerStock(stock *s, char nomFichier[]) {  
    FILE* fp = fopen(nomFichier, "rb");  
    produit p;  
    while (fread(&p;, sizeof(produit), 1, fp) == 1)  
        ajouterProduitFin(s, p);  
    fclose(fp);  
}
```

■ **Note:** `chargerStock` réutilise `ajouterProduitFin` — excellente démonstration de la modularité.

7. Points Clés & Erreurs Fréquentes

Erreur Fréquente	Explication & Solution
<code>=</code> vs <code>==</code>	<code>if(s->tete = NULL)</code> assigne au lieu de comparer. Toujours utiliser <code>==</code> .
<code>&arr</code> vs <code>arr</code>	Un tableau <code>char[]</code> est déjà une adresse. Ne pas écrire <code>&p.code</code> dans <code>scanf</code> .
<code>*code</code> vs <code>code</code>	Dans <code>strcmp</code> , passer <code>code</code> (<code>char*</code>) et non <code>*code</code> (premier caractère).
Variable non initialisée	<code>float sum;</code> contient une valeur garbage. Écrire <code>float sum = 0;</code>
Retour d'adresse locale	Ne jamais retourner <code>&variable_locale</code> (la pile est libérée au retour).
<code>fclose(filename)</code>	<code>fclose</code> prend un <code>FILE*</code> , pas un nom de fichier. <code>fclose(fp)</code> .
Double <code>fread</code>	Appeler <code>fread</code> dans la condition ET dans le corps de la boucle est une erreur.
supprimerFin — ptr nul	S'arrêter à <code>ptr->suivant->suivant != NULL</code> , pas <code>ptr->suivant != NULL</code> .

8. Conclusion

Ce projet couvre l'ensemble du cycle de vie d'une structure de données dynamique en C : définition des types, allocation mémoire, opérations CRUD, persistance fichier, et libération mémoire. Les concepts fondamentaux mis en pratique incluent :

- Les **listes chaînées** comme structure dynamique.
- La **gestion mémoire manuelle** avec `malloc` et `free`.
- La **modularité** pour éviter la duplication de code.
- Les **pointeurs sur heap** vs pointeurs sur pile.
- La **persistance binaire** avec `fread`/`fwrite` pour la sérialisation.